

Rozwiązywanie układów równań nieliniowych uproszczoną metodą Newtona

1 Zastosowanie

Procedura *SimplifiedNewton* rozwiązuje układ n równań nieliniowych uproszczoną metodą Newtona w zmien-nopozycyjnej arytmetyce klasycznej jak i przedziałowej postaci:

$$f_k(x_1, x_2, \dots, x_n) = 0, k = 1, 2, \dots, n$$

2 Opis metody

Układ równań nieliniowych (1) jest rozwiązywany uproszczoną wersją metody Newtona, w której kolejne przy-bliżenia rozwiązania są wyznaczane na podstawie następujących wzorów:

$$\begin{aligned}x_1^{(l+1)} &= x_1^{(l)} - \omega \frac{f_1(x_1^{(l)}, x_2^{(l)}, \dots, x_n^{(l)})}{\frac{\partial f_1}{\partial x_1}(x_1^{(l)}, x_2^{(l)}, \dots, x_n^{(l)})}, \\x_2^{(l+1)} &= x_2^{(l)} - \omega \frac{f_2(x_1^{(l+1)}, x_2^{(l)}, \dots, x_n^{(l)})}{\frac{\partial f_2}{\partial x_2}(x_1^{(l+1)}, x_2^{(l)}, \dots, x_n^{(l)})}, \quad l = 0, 1, \dots \\&\dots \\x_n^{(l+1)} &= x_n^{(l)} - \omega \frac{f_n(x_1^{(l+1)}, \dots, x_{n-1}^{(l+1)}, x_n^{(l)})}{\frac{\partial f_n}{\partial x_n}(x_1^{(l+1)}, \dots, x_{n-1}^{(l+1)}, x_n^{(l)})}\end{aligned} \quad (1)$$

gdzie $x_k^{(0)}$ ($k = 1, 2, \dots, n$) oznacza początkowe przybliżenie niewiadomej x_k , a ω – parametr procesu itera-cyjnego, taki że $0 < \omega < 2$ (zwykle przyjmuje się, że $\omega = 1$).

2.1 Kryteria stopu i kontrola zbieżności

Proces iteracyjny w obu zaimplementowanych procedurach sterowany jest dynamicznie za pomocą kryterium stopu opartego na zadanej dokładności ε (**eps**). Warunek zakończenia obliczeń realizowany jest sprzętowo na poziomie rejestru flag procesora:

Po zakończeniu pętli wewnętrznej, aktualna norma maksimum błędu przybliżeń porównywana jest z warto-ścią **eps** za pomocą instrukcji **fcomi**. Norma ta wyznaczana jest jako największa bezwzględna zmiana składowej wektora w danej iteracji:

$$\max_{k=1..n} |x_k^{(l+1)} - x_k^{(l)}| < \varepsilon \quad (2)$$

Jeśli wyznaczony błąd spełnia powyższą relację ($\max_error < \varepsilon$), instrukcja skoku warunkowego **jb end_loop** natychmiast przerywa wykonywanie pętli głównej, zwracając sterowanie do środowiska C++.

W przypadku braku osiągnięcia zbieżności numerycznej (np. dla układów rozbieżnych), bezpiecznikiem za-pobiegającym utknięciu programu w nieskończonej pętli jest m.in. licznik **mit**. Po wykonaniu maksymalnej zadeklarowanej liczby iteracji (np. 150), pętla zostaje przerwana awaryjnie.

3 Wywołanie procedury

SimplifiedNewton($n, x, f, df, omega, mit, eps, it$)

4 Dane

n – liczba równań układu (równa liczbie niewiadomych),

x – wskaźnik na tablicę, której elementy $x[i]$ ($i = 0, 1, 2, \dots, n - 1$) zawierają początkowe przybliżenia rozwią-zania,

f – wskaźnik na tablicę, której elementy $f[i]$ ($i = 0, 1, 2, \dots, n-1$) są procedurami języka C/C++, które na wskaźniku r zapisują wynik $f_i(x_0, x_1, \dots, x_{n-1})$, gdzie $x_0, x_1, \dots, x_{n-1}$ są elementami tablicy, na którą wskaźnik jest podany w parametrze x .

df – wskaźnik na tablicę, której elementy $df[i]$ ($i = 0, 1, 2, \dots, n-1$) są procedurami języka C/C++, które na wskaźniku r zapisują wynik $df_i(x_0, x_1, \dots, x_{n-1})$, czyli pochodnej $\frac{\delta f_i}{\delta x_i}$, gdzie $x_0, x_1, \dots, x_{n-1}$ są elementami tablicy, na którą wskaźnik jest podany w parametrze x .

$omega$ – wartość parametru procesu iteracyjnego (1) i (??).

mit – maksymalna dozwolona liczba iteracji

eps – bezwzględna dokładność rozwiązania (bezwzględna dokładność między dwoma kolejnymi przybliżeniami, albo szerokość przedziału w zależności od trybu, zob. 2.1)

Uwaga! Po wykonaniu procedury *SimplifiedNewton* wartości elementów tablicy x są zmienione.

5 Wyniki

x – zmodyfikowana tablica x z danych z przybliżonymi rozwiązaniami układu równań.

6 Inne parametry

it – wskaźnik do pamięci do zwrotu liczby wykonanych iteracji. Nie większa od mit

Procedura do zwracania błędów używa rejestrów kontrolnych FPU X87. W przypadku napotkania problemów funkcja podaje błąd dzielenia przez zero, albo nieprawidłowej operacji. Wtedy pod wynikami są wartości nieprzewidywalne. Pozostałe błędy są skutkiem normalnej pracy tej i innych procedur i należy je ignorować. Każdorazowo przed wywołaniem tej funkcji należy wyczyścić status błędów fpu korzystając z polecenia *fnclcx*, albo procedury *ClearX87Errors*.

7 Typy parametrów

```

long long      :  n, mit
long long*     :  it
double _num*   :  x, omega
extended_double* : eps
void(*f[])(double _num*, double _num*) : f, df

```

8 Identyfikatory nielokalne

extended_double – struktura 10 bajtowa (80-bitowa) pozwalająca na ominięcie ograniczenia *long double* do 64 bitów przez kompilator MSVC.

double_num – union 20 bajtowy typów *extended_double* i *interval*, będącego strukturą dwóch *extended_double*.

software_mode – enum typu C zawarty w poniższej bibliotece, który przełącza między trybami przedziałowymi i klasycznym

Biblioteka *math_core* będąca niezbędna do działania tej procedury implementuje powyższe struktury i udostępnia interfejs pozwalający wykonywać działania wykorzystując pełnię możliwości architektury X87.

Wszystkie funkcje i procedury użyte w tej dokumentacji i programie mają implementację w tej lub standardowej dla języka C/C++ bibliotece.

9 Tekst procedury

```

1  SimplifiedNewton proc
2  sub rsp, 40                ; Shadow space dla x64 ABI
3
4  mov rax, software_mode     ; Pobierz tryb (musi być EXTERN QWORD)
5  test rax, rax
6  jz 0F                     ; Skok do najbliższego @@ (Forward)
7

```

```

8      ; Tryb przedziałowy
9      call SimplifiedNewton_Point
10     jmp ending
11
12     @@:
13     ; Tryb punktowy (float80)
14     call SimplifiedNewton_Point
15
16     ending:
17     add rsp, 40
18     ret
19     SimplifiedNewton endp

```

9.1 SimplifiedNewton

```

1  SimplifiedNewton_Point proc
2      push rbx
3      push r12
4      push r13
5      push r14
6      push r15
7      push rsi
8      push rdi
9      push rbp
10     sub rsp, 160 ; Ustawienie stosu
11
12     mov rbx, rcx
13     mov r13, rdx
14     mov r14, r8
15     mov rbp, r9 ; Zapisanie zmiennych
16
17
18     mov r12, [rsp+160+8+8+8+8+8+8+32+8+8+48+8] ; mit
19     main_loop:
20     fldz
21     fstp tbyte ptr [rsp+96]
22     push rbx ; Bufor na maksymalny błąd iteracji
23
24     mov rsi, r14
25     mov rdi, rbp
26     mov r15, r13
27     iter_loop:
28     lea rcx, [rsp+48]
29     mov rdx, r13
30     mov rax, [rsi]
31     sub rsp, 32
32     call rax
33     add rsp, 32
34     add rsi, 8 ; f[i]
35
36     lea rcx, [rsp+72]
37     mov rdx, r13
38     mov rax, [rdi]
39     sub rsp, 32
40     call rax
41     add rsp, 32
42     add rdi, 8 ; df[i]
43
44     lea rcx, [rsp+48]
45     lea rdx, [rsp+72]
46     lea r8, [rsp+48]
47     call M_Div ; Liczenie poprawki
48
49     lea rcx, [rsp+48]
50     mov rdx, [rsp+160+8+8+8+8+8+8+32+8+8+48+8]; omega
51     lea r8, [rsp+48]
52     call M_Mul ; Mnożenie przez omegę
53
54     lea rcx, [r15]
55     lea rdx, [rsp+48]
56     lea r8, [r15]
57     call M_Sub ; Nałożenie poprawki
58
59     lea rcx, [rsp+48]
60     lea rdx, [rsp+126+8]
61     call M_Mid ; Łuskanie korekty
62
63     fld tbyte ptr [rsp+96+8]
64     fld tbyte ptr [rsp+126+8]
65     fabs
66     fcomi st(0), st(1)
67     fcmovb st(0), st(1)
68     fstp tbyte ptr [rsp+96+8]
69     fstp st(0) ; Zapisanie większej korekty
70
71     add r15, 20
72     dec rbx
73     jnz iter_loop

```

```

74     pop rbx
75     mov rdx, [rsp+160+8+8+8+8+8+8+32+8+8+48+8] ; eps
76     fld tbyte ptr [rdx]
77     fld tbyte ptr [rsp+96]
78     fcomi st(0), st(1) ; Porównanie z epsilon
79     fstp st(0)
80     fstp st(0)
81     jb end_loop
82     dec r12
83     jnz main_loop
84
85     end_loop:
86     mov r13, [rsp+160+8+8+8+8+8+8+32+8+8+48+8] ; mit
87     sub r13, r12
88     mov rcx, [rsp+160+8+8+8+8+8+8+32+8+8+48+8+8] ; it
89     mov [rcx], r13
90
91     add rsp, 160
92     pop rbp
93     pop rdi
94     pop rsi
95     pop r15
96     pop r14
97     pop r13
98     pop r12
99     pop rbx ; Sprzątanie stosu
100    ret
101    SimplifiedNewton_Point endp

```

10 Przykłady

a) Układ równań:

$$\begin{aligned}
 2x_1^2 + x_2 + x_3 &= 0 \\
 x_1x_2 - x_3 + 2 &= 0 \\
 x_1x_3 - x_2x_3 &= 0
 \end{aligned}$$

Definicje procedur f i df (w formacie eksportowym DLL):

```

1  const double_num c2 = {
2      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x40, // .low
3      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x40
4  };
5  const extended_double _4 = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x01, 0x40 };
6
7  // f1(x,y,z) = 2*x^2 + y + z = 0
8  __declspec(dllexport) void f1(double_num* ret, double_num* tab) {
9      double_num* x = &tab[0], * y = &tab[1], * z = &tab[2];
10     double_num x2, cx2, sum1;
11
12     M_PowInt(x, &c2.inter.high, &x2);
13     M_Mul(&c2, &x2, &cx2); // 2 * x^2
14     M_Add(&cx2, y, &sum1); // 2 * x^2 + y
15     M_Add(&sum1, z, ret); // 2 * x^2 + y + z
16 }
17
18 // f2(x,y,z) = x*y - z + 2 = 0
19 __declspec(dllexport) void f2(double_num* ret, double_num* tab) {
20     double_num* x = &tab[0], * y = &tab[1], * z = &tab[2];
21     double_num xy, sub1;
22
23     M_Mul(x, y, &xy); // x * y
24     M_Sub(&xy, z, &sub1); // x * y - z
25     M_Add(&sub1, &c2, ret); // x * y - z + 2
26 }
27
28 // f3(x,y,z) = x*z - y*z = 0
29 __declspec(dllexport) void f3(double_num* ret, double_num* tab) {
30     double_num* x = &tab[0], * y = &tab[1], * z = &tab[2];
31     double_num xz, yz;
32
33     M_Mul(x, z, &xz); // x * z
34     M_Mul(y, z, &yz); // y * z
35     M_Sub(&xz, &yz, ret); // x * z - y * z
36 }
37
38 // dF1/dx = 4x
39 __declspec(dllexport) void df1(double_num* ret, double_num* tab) {
40     double_num c4; M_LoadNum(&_4, &c4);
41     M_Mul(&c4, &tab[0], ret);
42 }
43
44 // dF2/dy = x
45 __declspec(dllexport) void df2(double_num* ret, double_num* tab) {

```

```

46         extended_double temp;
47         M_Mid(&tab[0], &temp);
48         M_LoadNum(&temp, ret);
49     }
50
51     // dF3/dz = x - y
52     __declspec(dllexport) void df3(double_num* ret, double_num* tab) {
53         M_Sub(&tab[0], &tab[1], ret);
54     }

```

Dane:

$n = 3$, $\omega = 1,0$, $\epsilon = 1E - 16$, $mit = 150$, $x[0] = 1,0$, $x[1] = -1,0$, $x[2] = -1,0$

Tryb: klasyczny

Zapisy z logów:

```

Loaded values:
x[0]: 1.00000000000000000000E+0
x[1]: -1.00000000000000000000E+0
x[2]: -1.00000000000000000000E+0
Running SimplifiedNewton in float80 mode. With variables: mit=150, eps
=9.99999999999999999999E-17, omega=1.00000000000000000000E+0.
Program has run through 54 iterations.
Results:
x[0]: 9.99999999999999999999E-1
x[1]: -2.00000000000000000000E+0
x[2]: 0.00000000000000000000E+0

```

b) Dla danych z a)

Dane:

$n = 3$, $\omega = 0,1$, $\epsilon = 1E - 5$, $mit = 10$, $x[0] = [1,0;1,01]$, $x[1] = [-1,0;-1,0]$, $x[2] = [-1,0;-1,0]$

Tryb: przedziałowy

Zapisy z logów:

```

Loaded values:
x[0]: 1.00499999999999999999E+0, [9.99999999999999999999E-1;1.00999999999999999999E+0]
x[1]: -1.00000000000000000000E+0, [-1.00000000000000000000E+0;-1.00000000000000000000E+0]
x[2]: -1.00000000000000000000E+0, [-1.00000000000000000000E+0;-1.00000000000000000000E+0]
Running SimplifiedNewton in pure interval mode. With variables: mit=10, eps
=9.99999999999999999999E-6, omega=1.00000000000000000000E-1.
Program has run through 10 iterations.
Results:
x[0]: 1.0665876213268528582E+0, [9.5562245866537851437E-1;1.1775527839883272020E+0], width
=2.21E-1
x[1]: -2.036586863007077393E+0, [-2.385950197662250735E+0;-1.687223528351904050E+0], width
=6.98E-1
x[2]: -4.201027447526779268E-1, [-9.015709837123699148E-1;6.1365494207014061096E-2], width
=9.62E-1

```

c) Dla danych z a)

Dane:

$n = 3$, $\omega = 0,1$, $\epsilon = 1E - 5$, $mit = 10$, $x[0] = 1,0$, $x[1] = -1,0$, $x[2] = -1,0$

Tryb: przedziałowy z rozszerzeniem wejść

Zapisy z logów:

```

Loaded values:
x[0]: 1.00000000000000000000E+0, [9.99999999999999999999E-1;1.00000000000000000001E+0]
x[1]: -1.00000000000000000000E+0, [-1.00000000000000000000E+0;-9.99999999999999999999E-1]
x[2]: -1.00000000000000000000E+0, [-1.00000000000000000000E+0;-9.99999999999999999999E-1]
Running SimplifiedNewton in interval float data mode. With variables: mit=10, eps
=9.99999999999999999999E-6, omega=1.00000000000000000000E-1.
Program has run through 10 iterations.
Results:
x[0]: 1.0649651535761333155E+0, [9.6548233208767718730E-1;1.1644479750645894438E+0], width
=1.98E-1
x[1]: -2.044494366865486012E+0, [-2.376924899193790550E+0;-1.712063834537181474E+0], width
=6.64E-1
x[2]: -4.252923894172914226E-1, [-9.016794588302957817E-1;5.1094679995712936415E-2], width
=9.52E-1

```

d) Dla danych z a)

Dane:

$n = 3$, $\omega = 0,1$, $\epsilon = 1E-5$, $mit = 100$, $x[0] = [1,0;1,0]$, $x[1] = [-1,0;-1,0]$, $x[2] = [-1,0;-1,0]$

Tryb: przedziałowy

Zapisy z logów:

```
Loaded values:
x[0]: 1.0000000000000000E+0, [9.999999999999998E-1;1.0000000000000000E+0]
x[1]: -1.0000000000000000E+0, [-1.0000000000000000E+0;-1.0000000000000000E+0]
x[2]: -1.0000000000000000E+0, [-1.0000000000000000E+0;-1.0000000000000000E+0]
Running SimplifiedNewton in pure interval mode. With variables: mit=100, eps
=9.9999999999999970E-6, omega=1.0000000000000000E-1.
There were some critical errors, or Newton was not convergent. Try with different data. Error
code: 0x2d.
```